

[Algorithm]

스택 및 큐 응용

인하대학교 소프트웨어융합공학과

김종훈 교수



인하대학교
INHA UNIVERSITY

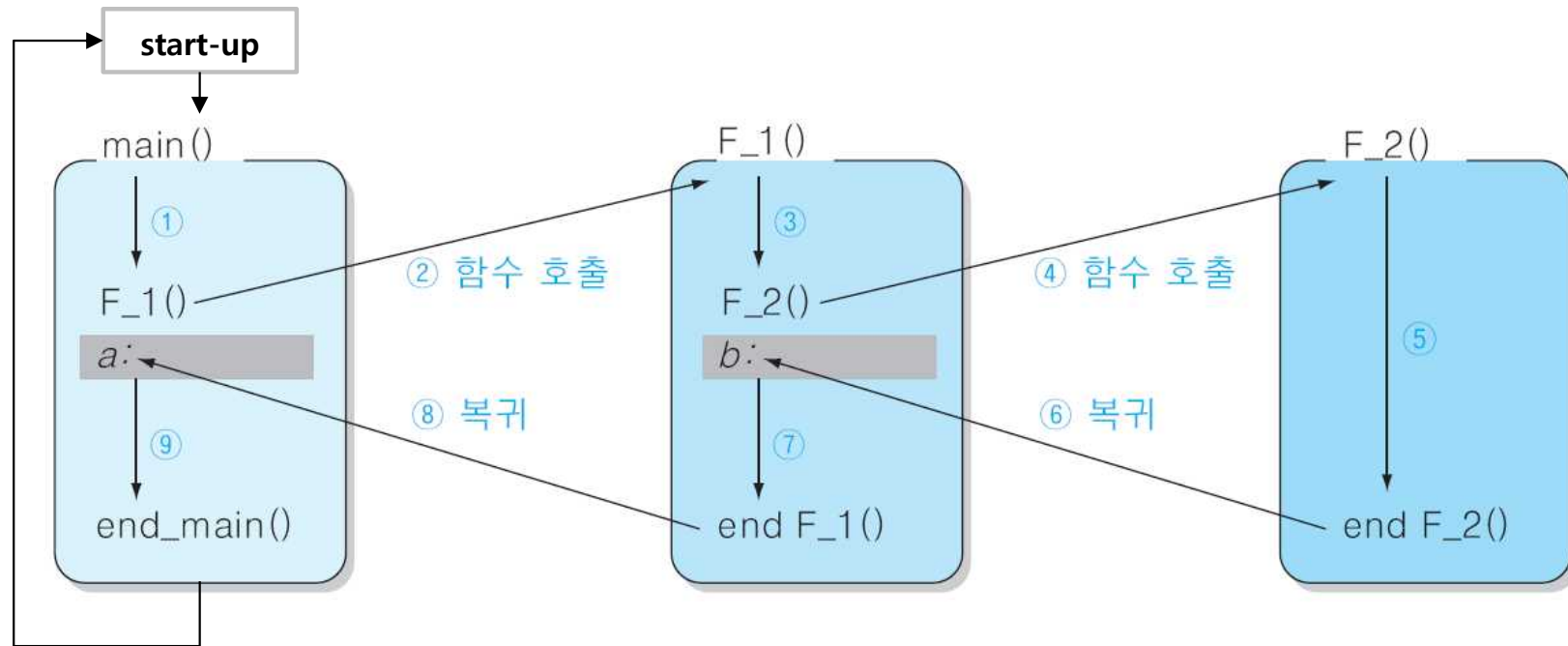
❖ 시스템 스택

■ 프로그램에서의 호출과 복귀에 따른 수행 순서를 관리

- 가장 마지막에 호출된 함수가 가장 먼저 실행을 완료하고 복귀하는 후입 선출 구조이므로, 후입선출 구조의 스택을 이용하여 수행순서 관리
- 함수 호출이 발생하면 호출한 함수 수행에 필요한 지역변수, 매개변수 및 수행 후 복귀할 주소 등의 정보를 스택 프레임(stack frame)에 저장하여 시스템 스택에 삽입
- 함수의 실행이 끝나면 시스템 스택의 top 원소(스택 프레임)를 삭제(pop)하면서 프레임에 저장되어있던 복귀주소를 확인하고 복귀
- 함수 호출과 복귀에 따라 이 과정을 반복하여 전체 프로그램 수행이 종료되면 시스템 스택은 공백스택이 된다.

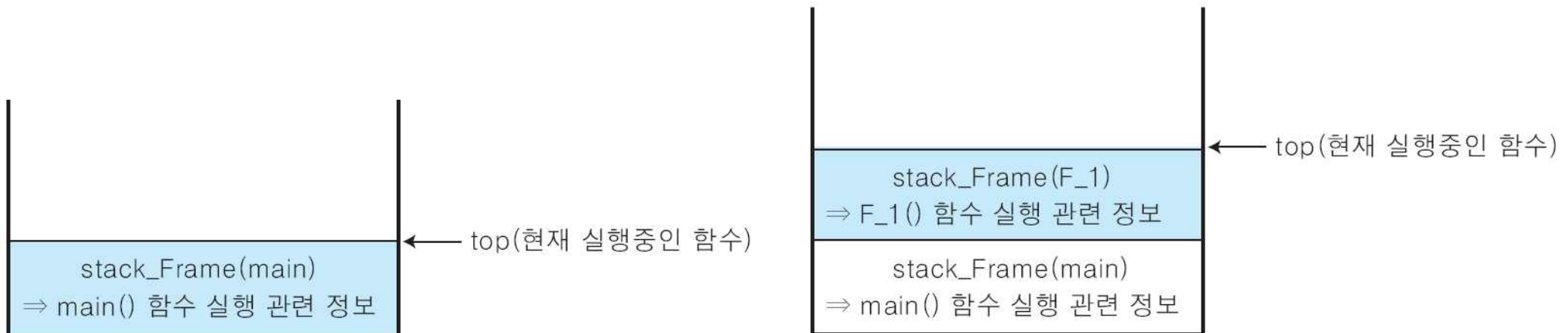
□ 스택의 응용 - 시스템 스택

- 함수 호출과 복귀에 따른 전체 프로그램의 수행 순서



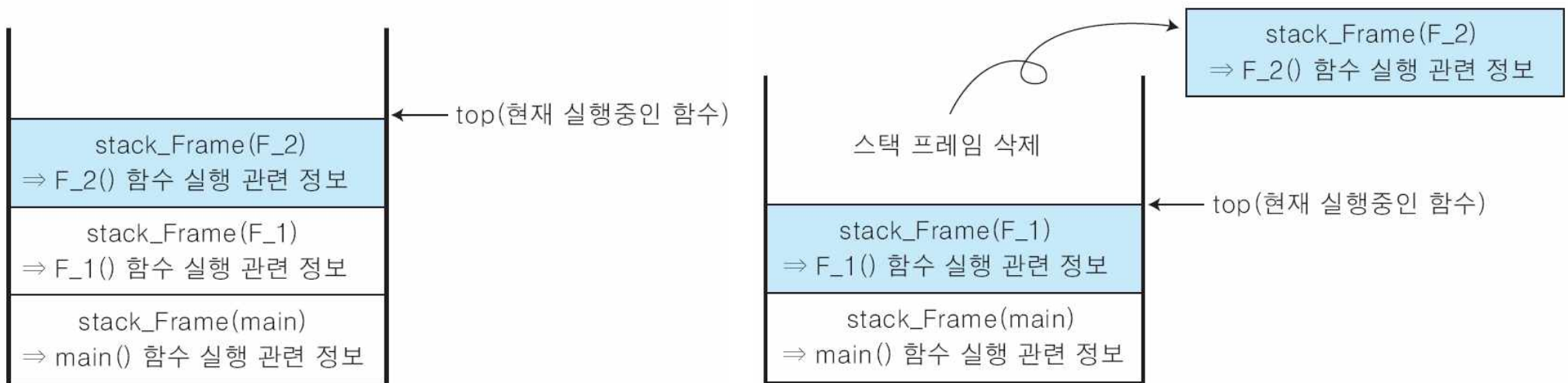
□ 스택의 응용 - 시스템 스택

- ① 프로그램이 실행을 시작하여 main() 함수가 실행되면서 main() 함수에 관련된 정보를 스택 프레임에 저장하여 시스템 스택에 삽입
- ② main() 함수 실행 중에 F_1() 함수 호출을 만나면 함수 호출과 복귀에 필요한 정보를 스택 프레임에 저장하여 시스템 스택에 삽입하고, 호출된 함수인 F_1() 함수로 이동
 - 이때 스택 프레임에는 호출된 함수의 수행이 끝나고 main() 함수로 복귀할 주소 a를 저장



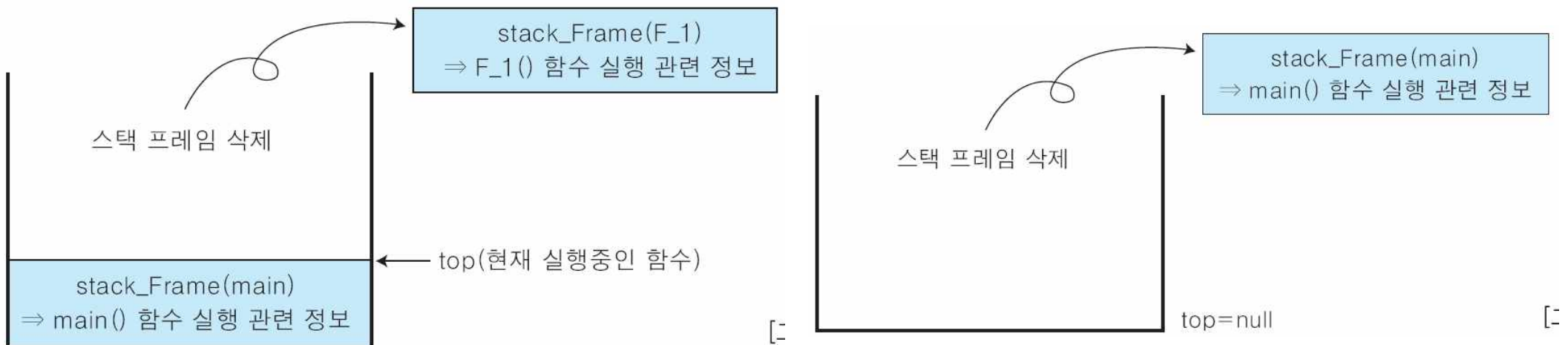
□ 스택의 응용 - 시스템 스택

- ③ 호출된 함수 $F_1()$ 함수를 실행
- ④ $F_1()$ 함수 실행 중에 $F_2()$ 함수 호출을 만나면 다시 함수 호출과 복귀에 필요한 정보를 스택 프레임에 저장하여 시스템 스택에 삽입. 호출된 함수인 $F_2()$ 함수를 실행
 - 스택 프레임에는 $F_1()$ 함수로 복귀할 주소 b 를 저장
- ⑤ 호출된 함수 $F_2()$ 함수를 실행 완료
- ⑥ 시스템 스택의 top에 있는 스택 프레임을 pop하여 정보를 확인하고, $F_1()$ 함수의 b 주소로 복귀



□ 스택의 응용 - 시스템 스택

- ⑦ 복귀된 함수 F_1() 함수의 b주소 이후 부분을 실행
- ⑧ 시스템 스택의 top에 있는 스택 프레임을 pop하여 정보를 확인하고 main() 함수의 a주소로 복귀
- ⑨ 복귀된 main() 함수의 a주소 이후 부분에 대한 실행이 완료되면 시스템 스택의 top에 있는 스택 프레임을 pop하는데, 시스템 스택이 공백이 되었으므로 전체 프로그램 실행 종료



□ 스택의 응용 - 수식의 괄호 검사

❖ 수식의 괄호 검사

- 수식에 포함되어있는 괄호는 가장 마지막에 열린 괄호를 가장 먼저 닫아 주어야 하는 후입선출 구조로 구성되어있으므로, 후입선출 구조의 스택을 이용하여 괄호를 검사한다.
- 수식을 왼쪽에서 오른쪽으로 하나씩 읽으면서 괄호 검사
 - 왼쪽 괄호를 만나면 스택에 push
 - 오른쪽 괄호를 만나면 스택을 pop하여 마지막에 저장한 괄호와 같은 종류인지를 확인
 - 같은 종류의 괄호가 아닌 경우 괄호의 짝이 잘못 사용된 수식!
 - 수식에 대한 검사가 모두 끝났을 때 스택은 공백 스택
 - 수식이 끝났어도 스택이 공백이 되지 않으면 괄호의 개수가 틀린 수식!

□ 스택의 응용 - 수식의 괄호 검사

■ 수식의 괄호 검사 알고리즘

```
testPair( )  
  exp ← Expression;  
  Stack ← null;  
  while (true) do {  
    symbol ← getSymbol(exp);  
    case {  
      symbol = "(" or "[" or "{" :  
        push(Stack, symbol);  
      symbol = ")" :  
        open_pair ← pop(Stack);  
        if (open_pair ≠ "(") then return false;  
      symbol = "]" :  
        open_pair ← pop(Stack);  
        if (open_pair ≠ "[") then return false;  
      symbol = "}" :  
        open_pair ← pop(Stack);  
        if (open_pair ≠ "{") then return false;
```


□ 스택의 응용 - 수식의 괄호 검사

■ 수식의 괄호 검사 알고리즘

```
        symbol = null :  
            if (isEmpty(Stack)) then return true;  
            else return false;  
        else :  
            }  
        }  
end testPair( )
```

□ 스택의 응용 - 수식의 괄호 검사

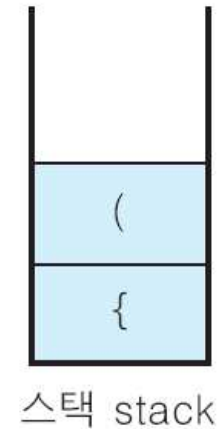
■ 수식의 괄호 검사 예

- 검사할 수식 : $\{(A+B)-3\} * 5 + [\{\cos(x+y)+7\}-1] * 4$

$\{(A+B)-3\} * 5 + [\{\cos(x+y)+7\}-1] * 4$

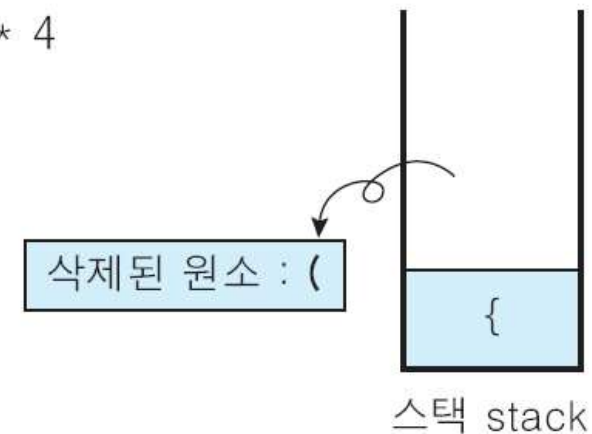
① push(stack, {)

② push(stack, ()



$\{(A+B)-3\} * 5 + [\{\cos(x+y)+7\}-1] * 4$

③ pop(stack)

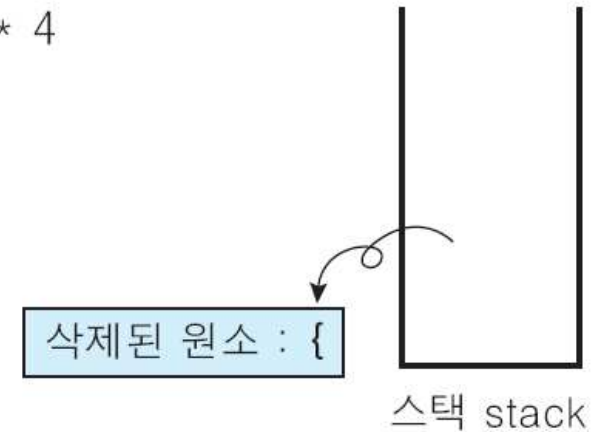


>> 계속

□ 스택의 응용 - 수식의 괄호 검사

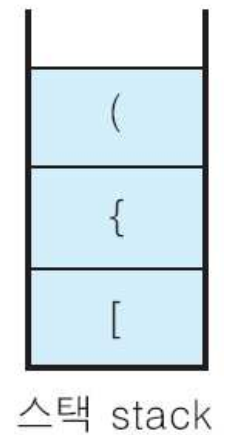
$\{ (A + B) - 3 \} * 5 + [\{ \cos(x + y) + 7 \} - 1] * 4$

④ pop(stack)



$\{ (A + B) - 3 \} * 5 + [\{ \cos(x + y) + 7 \} - 1] * 4$

⑤ push(stack, [)
⑥ push(stack, {)
⑦ push(stack, ()



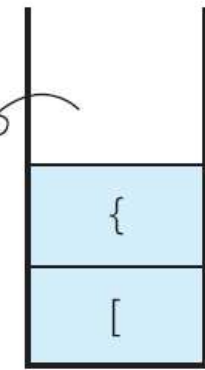
>> 계속

□ 스택의 응용 - 수식의 괄호 검사

$\{ (A + B) - 3 \} * 5 + [\{ \cos(x + y) + 7 \} - 1] * 4$

⑧ pop(stack)

삭제된 원소 : (

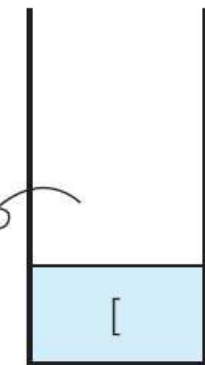


스택 stack

$\{ (A + B) - 3 \} * 5 + [\{ \cos(x + y) + 7 \} - 1] * 4$

⑨ pop(stack)

삭제된 원소 : {



스택 stack

□ 스택의 응용 - 수식의 괄호 검사

$\{ (A + B) - 3 \} * 5 + [\{ \cos(x + y) + 7 \} - 1] * 4$

⑩ pop(stack)

삭제된 원소 : [

스택 stack

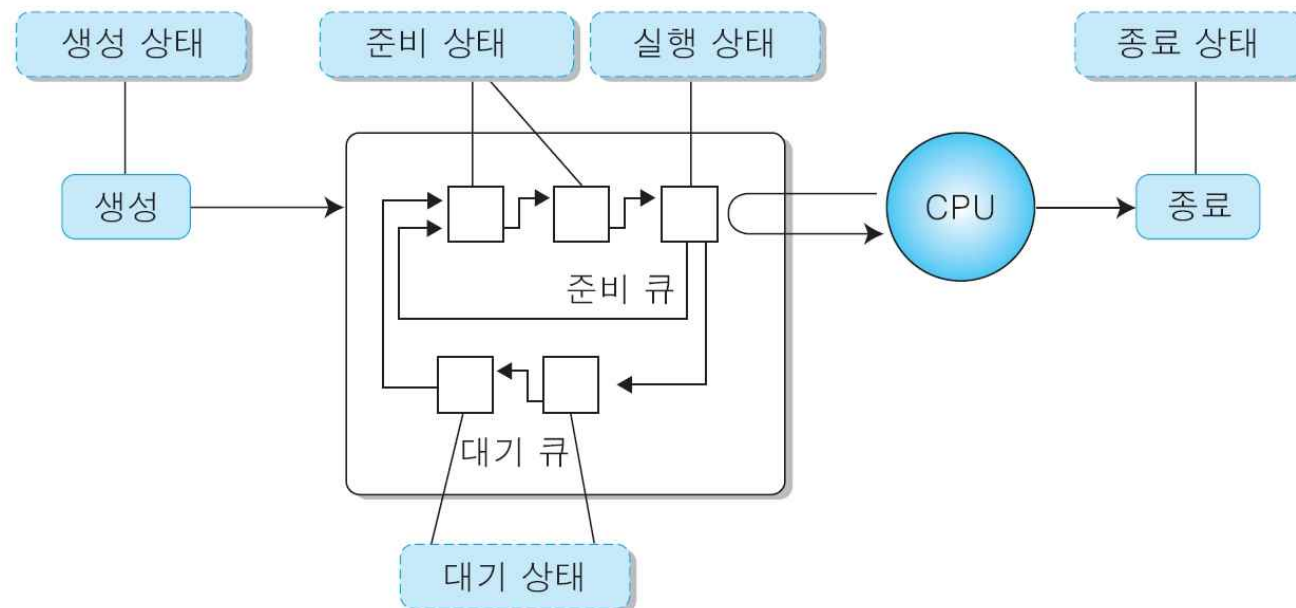
❖ 운영체제의 작업 큐

■ 프린터 버퍼 큐

- CPU에서 프린터로 보낸 데이터 순서대로(선입선출) 프린터에서 출력하기 위해서 선입선출 구조의 큐 사용

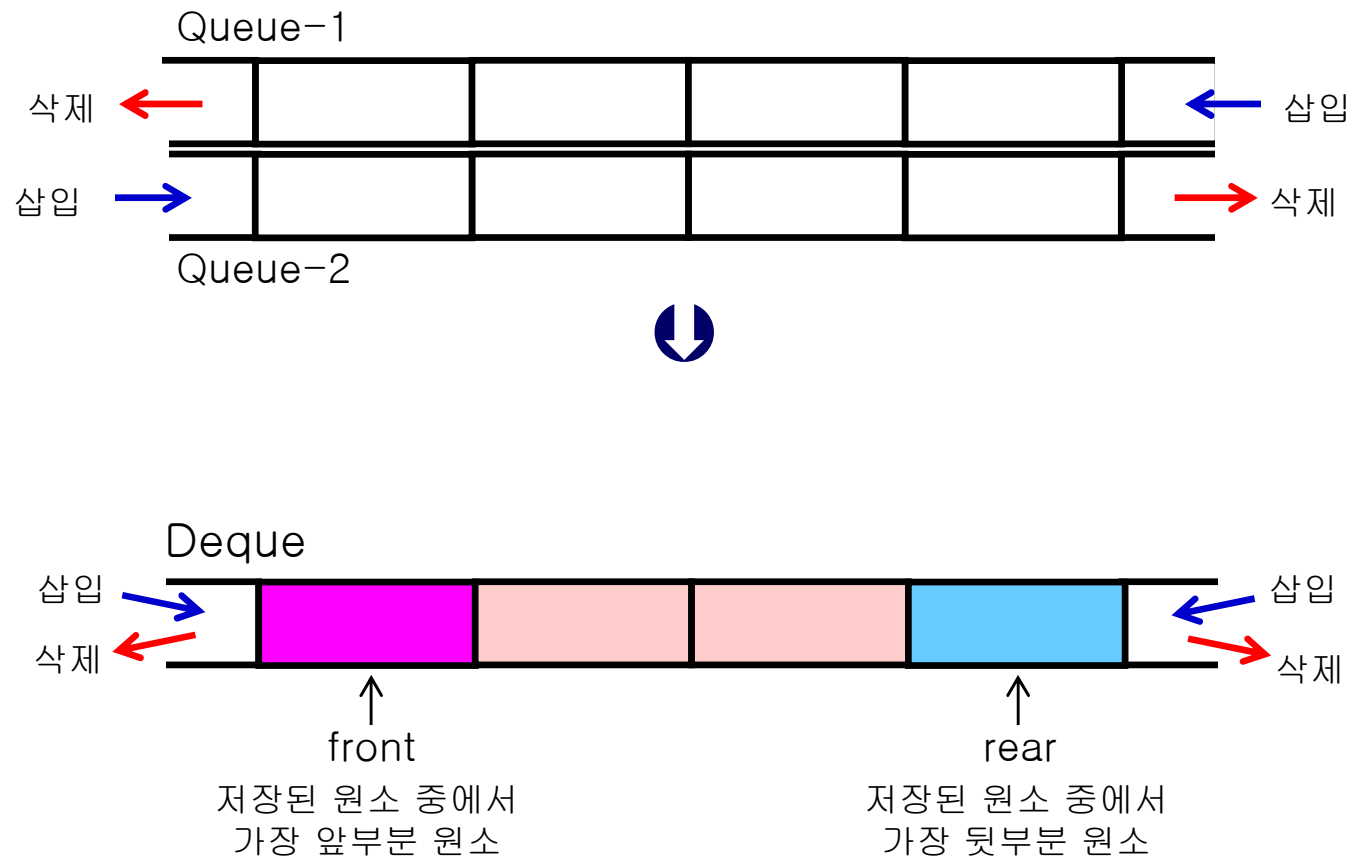
■ 스케줄링 큐

- CPU 사용을 요청한 프로세스들의 순서를 스케줄링하기 위해 큐를 사용



❖ 덱(Deque, double-ended queue)

- 큐 2개를 반대로 붙여서 만든 자료구조



■ 덱에 대한 추상 자료형

ADT deque

[ADT 8-2]

데이터 : 0개 이상의 원소를 가진 유한 순서 리스트

연산 :

$DQ \in \text{deque}; \text{item} \in \text{Element};$

$\text{createDeque}() ::= \text{create an empty DQ};$

// 공백 덱을 생성하는 연산

$\text{isEmpty}(DQ) ::= \text{if } (DQ \text{ is empty}) \text{ then return true}$

else return false;

// 덱이 공백인지 아닌지를 확인하는 연산

$\text{insertFront}(DQ, \text{item}) ::= \text{insert item at the front of DQ};$

// 덱의 front 앞에 item(원소)을 삽입하는 연산

$\text{insertRear}(DQ, \text{item}) ::= \text{insert item at the rear of DQ};$

// 덱의 rear 뒤에 item(원소)을 삽입하는 연산

$\text{deleteFront}(DQ) ::= \text{if } (\text{isEmpty}(DQ)) \text{ then return null}$

else { delete and return the front item of DQ };

// 덱의 front에 있는 item(원소)을 덱에서 삭제하고 반환하는 연산

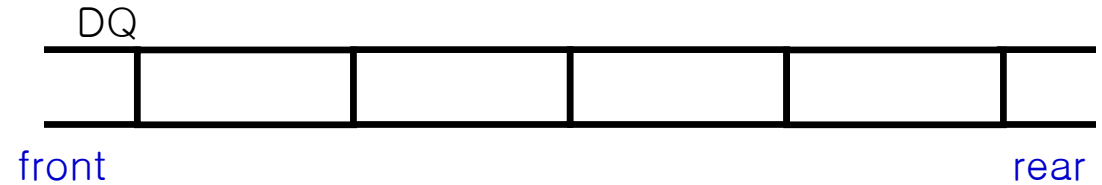
$\text{deleteRear}(DQ) ::= \text{if } (\text{isEmpty}(DQ)) \text{ then return null}$

else { delete and return the rear item of DQ };

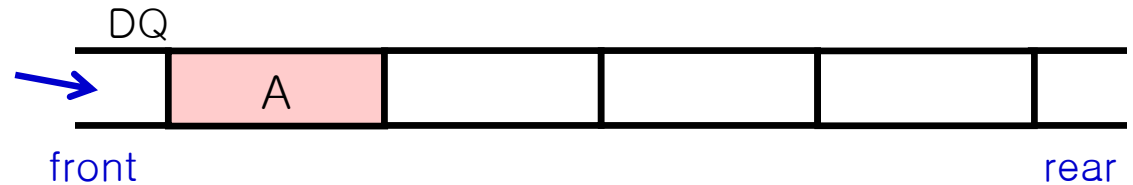

```
// 덱의 rear에 있는 item(원소)을 덱에서 삭제하고 반환하는 연산
removeFront(DQ) ::= if (isEmpty(DQ)) then return null
                    else { remove the front item of DQ };
// 덱의 front에 있는 item(원소)을 삭제하는 연산
removeRear(DQ) ::= if (isEmpty(DQ)) then return null
                  else { remove the rear item of DQ };
// 덱의 rear에 있는 item(원소)을 삭제하는 연산
peekFront(DQ) ::= if (isEmpty(DQ)) then return null
                  else { return the front item of the DQ };
// 덱의 front에 있는 item(원소)을 반환하는 연산
peekRear(DQ) ::= if (isEmpty(DQ)) then return null
                 else { return the rear item of the DQ };
// 덱의 rear에 있는 item(원소)을 반환하는 연산
End deque
```

■ 덱에서의 연산 과정

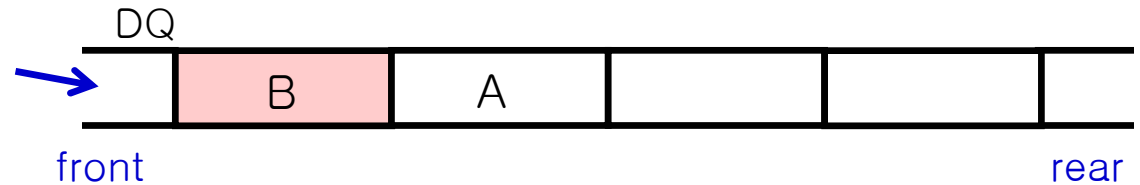
① createDeque();



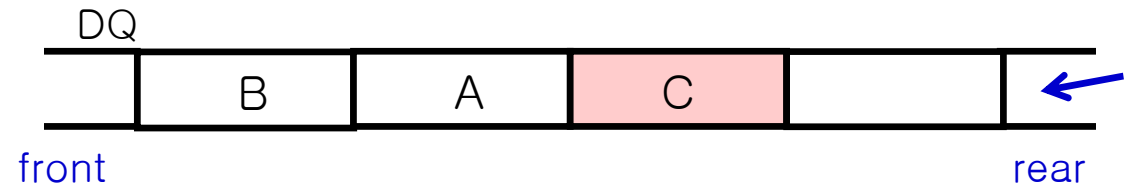
② insertFront(DQ, 'A');



③ insertFront(DQ, 'B');

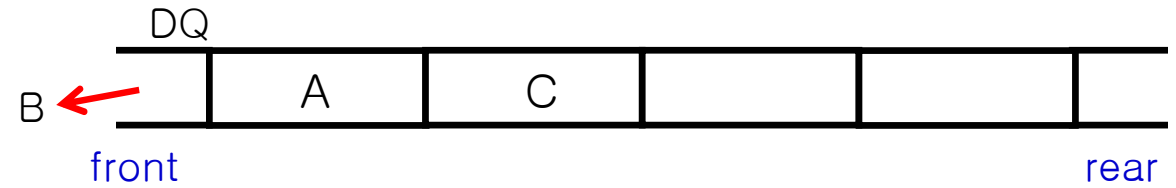


④ insertRear(DQ, 'C');

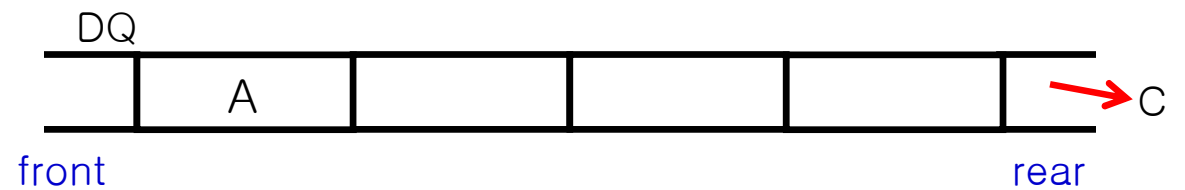




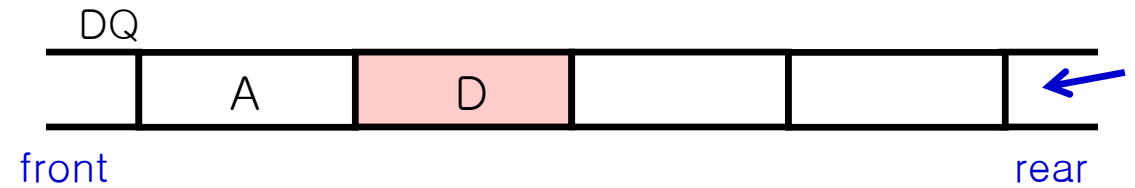
⑤ deleteFront(DQ);



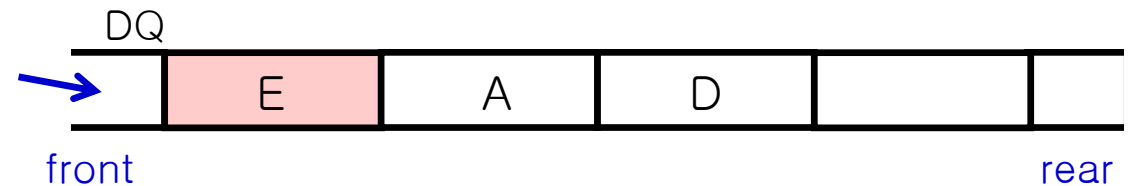
⑥ deleteRear(DQ);



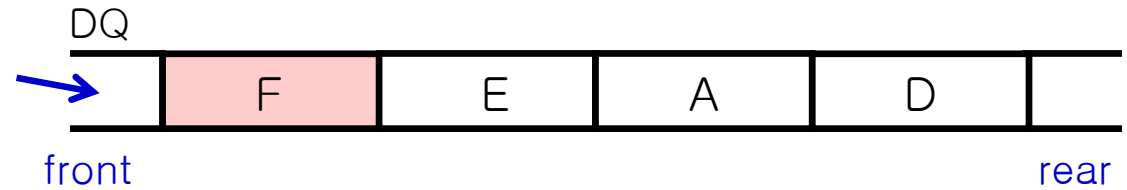
⑦ insertRear(DQ, 'D');



⑧ insertFront(DQ, 'E');

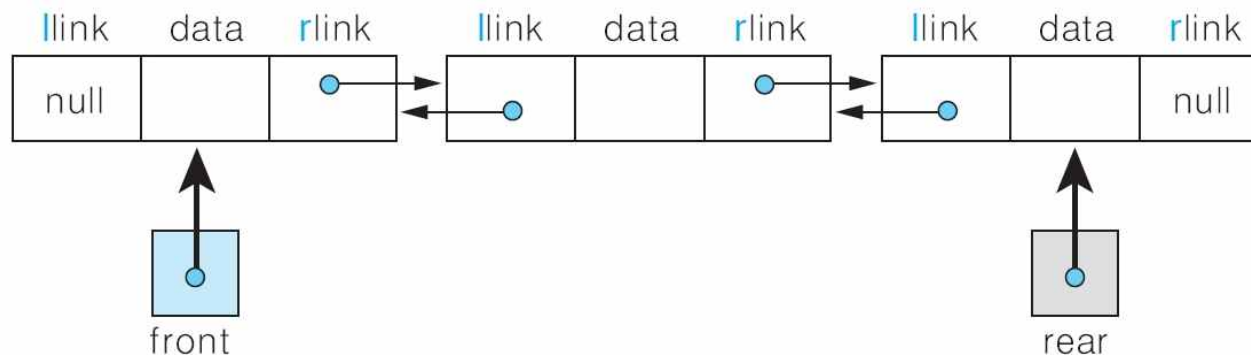


⑨ insertFront(DQ, 'F');



■ 덱의 구현

- 양쪽 끝에서 삽입/삭제 연산을 수행하면서 크기 변화와 저장된 원소의 순서 변화가 많으므로 순차 자료구조는 비효율적
- 양방향으로 연산이 가능한 이중 연결 리스트를 사용한다.



❖ 회문(回文, palindrome) 찾기 문제

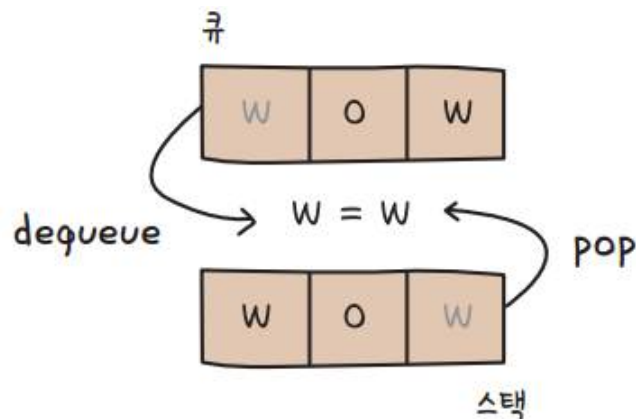
- 회문: 순서대로 읽어도 거꾸로 읽어도 그 내용이 같은 낱말이나 문장
- 낱말 사이에 있는 공백이나 문장 기호 등은 무시하므로 다음 낱말과 문장은 모두 회문

회문의 예(한글)	회문의 예(영어)
역삼역	mom
기러기	wow
일요일	noon
사진사	level
복불복	radar
다가가다	kayak
기특한 특기	racecar
다했냐? 했다!	God's dog
다시 합창 합시다	Madam, I'm Adam.

□ 회문 찾기

❖ 회문 찾기 알고리즘

- 주어진 문장의 문자들을 하나하나 큐와 스택에 넣은 다음 큐와 스택에서 하나씩 자료를 꺼낸다고 생각해 보자
- 큐는 들어간 순서 그대로, 스택은 들어간 순서와 정반대로 문자들이 나옴
- 회문은 거꾸로 읽어도 같은 글자가 나와야 함
- 따라서 큐에서 꺼낸 문자들(원래 순서)이 스택에서 꺼낸 문자들(역순)과 모두 같다면 그 문장은 회문



❖ 리스트로 큐와 스택 사용하기

- 큐와 스택은 자료를 일렬로 보관하는 특징이 있음
- 따라서 파이썬의 리스트를 이용해서 쉽게 만들어 볼 수 있음
- 아래표와 같은 방식으로 리스트를 사용해서 큐와 스택을 만들어 보자

자료 구조	동작	코드	설명
큐	초기화	<code>qu = []</code>	빈 리스트를 만들
	자료 넣기(enqueue)	<code>qu.append(x)</code>	리스트의 맨 뒤에 자료를 추가
	자료 꺼내기(dequeue)	<code>x = qu.pop(0)</code>	리스트의 맨 앞(0번 위치)에서 자료를 꺼냄
스택	초기화	<code>st = []</code>	빈 리스트를 만들
	자료 넣기(push)	<code>st.append(x)</code>	리스트의 맨 뒤에 자료를 추가
	자료 꺼내기(pop)	<code>x = st.pop()</code>	리스트의 맨 뒤에서 자료를 꺼냄

리스트로 큐와 스택 만들기

❖ 회문 찾기 알고리즘 구현

```
# 주어진 문장이 회문인지 아닌지 찾기(큐와 스택의 특징을 이용)
# 입력: 문자열 s
# 출력: 회문이면 True, 아니면 False

def palindrome(s):
    # 큐와 스택을 리스트로 정의
    qu = []
    st = []
    # 1단계: 문자열의 알파벳 문자를 각각 큐와 스택에 넣음
    for x in s:
        # 해당 문자가 알파벳이면(공백, 특수문자, 숫자가 아니면)
        # 큐와 스택에 각각 그 문자를 추가
        if x.isalpha():
            qu.append(x.lower())
            st.append(x.lower())
    # 2단계: 큐와 스택에 들어 있는 문자를 꺼내면서 비교
    while qu: # 큐에 문자가 남아 있는 동안 반복
        if qu.pop(0) != st.pop(): # 큐와 스택에서 꺼낸 문자가 다르면 회문이 아님
            return False

    return True

print(palindrome("Wow"))
print(palindrome("Madam, I' m Adam."))
print(palindrome("Madam, I am Adam."))
```




Thank You